Practical Multi-Repo Agentic Engineering

Björn Jarisch

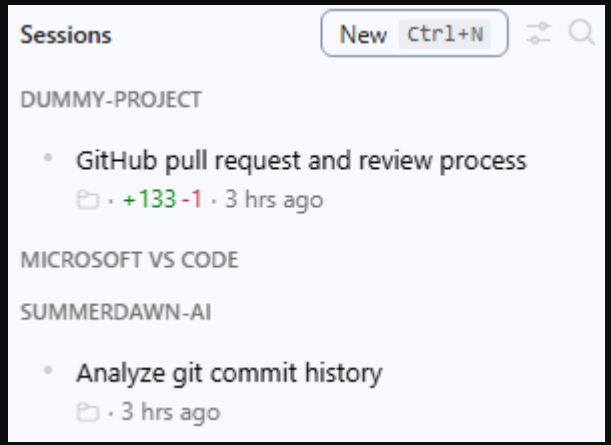
Software Architect & Engineer | Cloud & AI

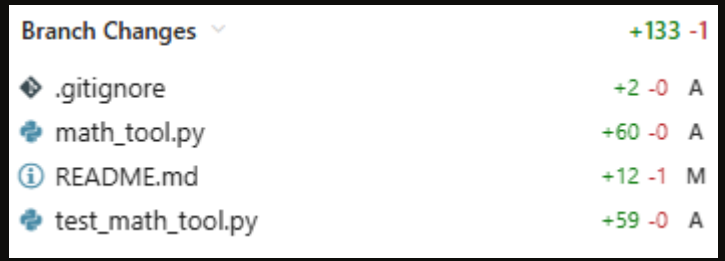
me@summerdawn.ai

TIP 1

Use the Agents Window

Use the new VS Code Agents window to see files and changes for each agent session





TIP 2

Automate repo creation with the GitHub CLI

Create, configure and initialize new repositories based on project requirements

> Create a new private repository called "dummy-project" in der summerdawn-ai organization using the GitHub CLI, make sure wiki and projects are disabled, then check it out locally, copy the latest version of the C# AGENTS.md file from the templates repo into it, and push everything to GitHub.

Done. I created the private GitHub repository at https://github.com/summerdawn-ai/dummy-project, cloned it locally to `c:\repos\summerdawn-ai\dummy-project`, added the AGENTS.md file from templates, and pushed everything to `main`.

TIP 3

Iterate on PRs using the GitHub MCP Server

See open PRs, start code reviews, resolve comments, repeat until done

Create a pull request for the current branch, then start a code review by Copilot and wait until it’s done? Fix any code review comments in separate commits if you think they're relevant, mark the comments as resolved, push your fixes. and request a new review. Keep going until we have a version with no comments.

The branch is pushed, the review threads are resolved, and the most recent Copilot review completed successfully with **no new code review comments**.

TIP 4

Keep agent instructions aligned

Compare AGENTS.md and other instructions across repos and apply changes made in one repository to other relevant repos

> Look at the AGENTS.md files in each repository in this folder, compare them, and list any instructions from one repository that should be added to the other repositories because they are universal.

The strongest pattern is that `kagami` and `mcpifier` share the same base **build/style hygiene** rules, while `summerdawn-private` has the richer **C# coding-style** rules, and `mcpifier` has the richest **comment/documentation** rules. These are the most plausible cross-repo distribution candidates: [...]

TIP 5

Analyze Git history to improve agent instructions

Look at the entire commit history, find the missteps, and distill AGENTS.md instructions that would have helped to avoid them

> Using subagents, look at the entire git commit history in each repository and identify situations where we had to do work that could have been avoided if we had better agent instructions, for example, refactorings or bug fixes. This applies both to follow-up commits within a branch and to complete branches. Provide a list of proposed new instructions and describe the problems that would have been avoided if we had had them in place.

The main pattern is that both repos already had style/scope guidance, but they were missing stronger instructions for **architecture-first work, validation/test discipline, and merge/rework prevention**.

Recommended AGENTS.md additions
[...]

BONUS TIP

Use the Speech extension to talk to your agents

Use the VS Code Speech extension to enable voice recognition in the Chat sidebar, or Windows 11 voice typing in the Agents window

Thank you.

Björn Jarisch

Software Architect & Engineer | Cloud & AI

me@summerdawn.ai